

Day2 · 13차시

# 계획·반성 루프 & 내 MVP에 통합

스스로 계획하고, 스스로 고쳐 쓰는 에이전트로.  
day2에서 모은 부품(도구·RAG·메모리·멀티)을 하나의 똑똑한 MVP에 끼우는 날.

SECTION 00

# 오프닝 — 부품을 모았으니, 이제 합친다

7~12차시에 배운 조각들을 day1 MVP 한 곳에 끼운다

**“에이전트가 한 번에 못 푸는 일을 — 어떻게 끝까지 해내게 할까?”**

— 답의 두 축: 계획(나눠서) + 반성(고쳐서)

## 어제까지 모은 부품들 — 지금 여기

▶ 오늘은 새 부품이 아니라, 모은 부품들 내 앱에 끼우는 날

1

### 7 추론·루프 · 8 설계

두뇌(Reasoning)와 ReAct 루프, 무엇을 맡길지 스코핑

2

### 9 도구·MCP · 10 메모리

세상과 상호작용하는 손발, 과거를 기억

3

### 11 RAG · 12 멀티·하네스

내 데이터에 근거, 여러 에이전트·실행 환경

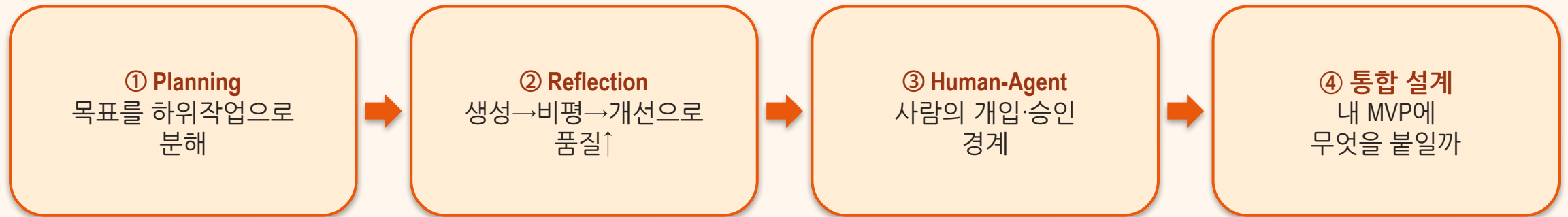
4

### 13 통합 (오늘)

흩어진 부품들 내 MVP 한 조각에 끼운다 → 똑똑해진 MVP

## 이 차시 동안 얻는 것

▶ 모든 내용이 마지막의 '내 팀 MVP에 적용'으로 수렴한다



SECTION 01

# Planning — 큰 일을 나눈다

목표 → 하위작업으로 쪼개야 다음에 무엇을 할지 안다

## 계획이 없으면 다음에 뭘 할지 모른다

▶ “계획이 없으면 에이전트가 다음에 무엇을 할지 어떻게 알겠는가?” — multi-step 존재의 필수품

1

### 다음 행동의 근거

여러 단계 중 지금 무엇을 할지 고르는 기준

2

### 진행 상황의 그릇

어디까지 왔는지 상태를 담아 추적

3

### 동적 수정의 발판

새 정보가 생기면 계획을 갱신할 수 있다

## “코드베이스에 기능 추가” — 계획으로 풀면

▶ 포인트: 어떤 행동을 해야 하는지 인지 + 어디까지 왔는지 추적

1

### 요구사항 명확화

어떤 기능이 필요한가 — 모호함을 먼저 제거

2

### 기존 코드 분석 → ③ 설계

아키텍처 이해(도구 호출) 후 빠짐없이 설계

3

### 구현 → ⑤ 테스트

동작을 깨지 않고 구현, 단위·통합 테스트로 검증

4

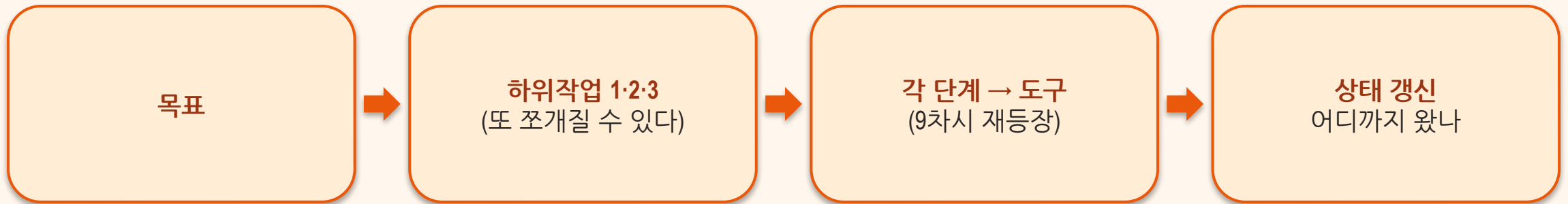
### 문서 갱신

새 기능 동작을 설명 — 진행 상태를 추적하며 완수



## Task Decomposition — 계층 구조

▶ 작업은 또 하위작업을 갖고, 각 작업은 도구를 가리킨다 — 계획이 먼저, 도구가 나중



## 그냥 생각하기 vs 계획 짜고 실행하기

▶ 실습 1단계가 Plan-and-Solve 패턴

### CoT (단계적 사고)

- “step-by-step 으로 생각해”
- 단계적 사고 그 자체가 분해
- 추론 흔적을 이어감

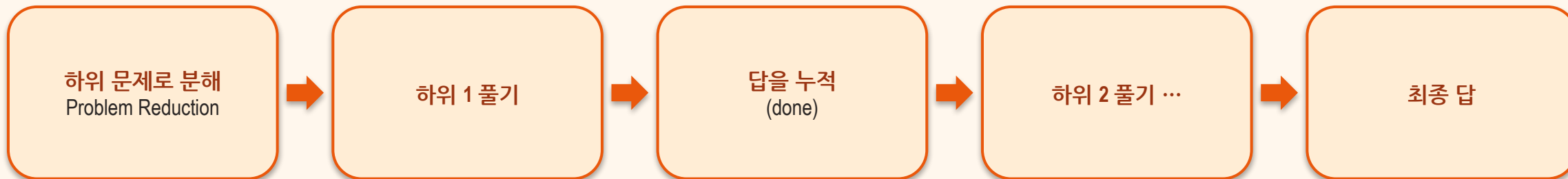
VS

### Plan-and-Solve

- “먼저 계획 → 그 계획대로 실행”
- 생각 단계 → 답 단계, **2단계**
- 계획이 명시적으로 남는다

## 쉬운 것부터 — 앞 단계의 답을 다음에 누적

▶ 실습 1단계의 done 변수가 바로 이 누적 컨텍스트



☞ CoT와의 차이 — 앞 단계의 답을 다음 프롬프트에 다시 주입, 쌓일수록 안정·정확

## 미리 다 계획 vs 하면서 계획 (ReAct)

▶ 안정된 작업 = 미리 계획(싸고 예측 가능) / 탐색적 작업 = ReAct(유연하지만 비쌈)

### Plan-then-Execute — 미리 계획

- 계획 1번(비싼 모델) → 실행 N번(싼 모델)
- 빠르고 저렴 · 예측 가능
- 중간에 상황이 바뀌면 취약

VS

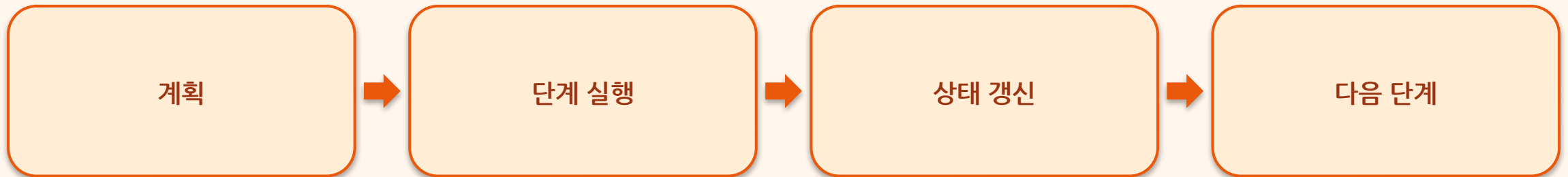
### Interleaved — ReAct처럼

- 매 스텝마다 생각→행동→관찰
- 새 정보에 즉시 적응
- 호출마다 비용 — 총비용↑

계획은 살아있다

## 동적 갱신 & Planner-Executor

▶ 큰 모델은 계획, 작은 모델은 실행 → 디버깅·비용 유리 (12차시 아키타입)



🔄 새 정보가 생기면 → 재계획 (malleable)

## 한 줄기가 불안하면 — 여러 갈래로 생각한다

▶ 여러 번 생각하게 하는 것도 결국 test-time compute — 7차시와 연결

### CoT — 한 줄기

한 번의 단계적 사고  
빠르지만 한 번 틀리면 끝

### Self-Consistency

같은 문제를 여러 번 풀어  
다수결로 답 선택

### Tree-of-Thoughts

사고를 가지로 뺏고  
평가·가지치기하며 탐색

## SECTION 02

# Reflection 루프 — 고쳐 쓰며 품질↑

생성 → 자기비평 → 개선. 어중간한 답(지역 최솟값)에서 빠져나온다

## 무엇을 더 개선할 수 있나 — 내부 과정

▶ 사람이 초안을 검토·수정하듯, 모델이 자기 출력을 비판한다

무엇

“지금까지 무엇을 했고, 무엇을 더 개선할 수 있나”를 되돌아보는 내부 과정

계획과의 관계

계획과 반성은 짝(feedback loop) — 함께 돌며 서로를 보완

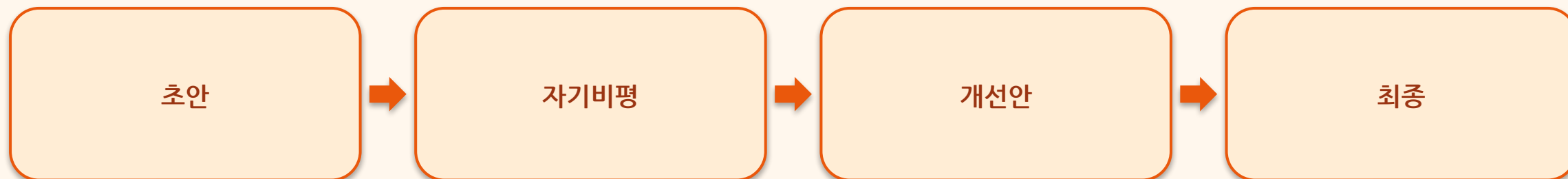
왜

어중간한 답(지역 최솟값, local minima)에 갇히는 걸 막는다



## 한 LLM이 작가이자 편집자

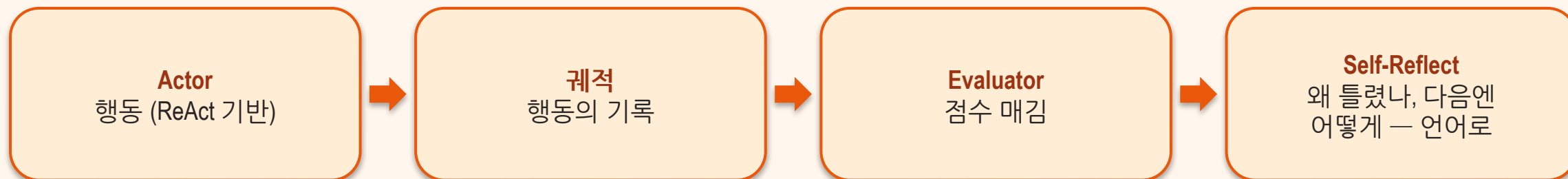
▶ 추가 학습·인프라 없이 프롬프팅만으로 품질↑ — 실습 2단계



🔄 부족하면 반복 — 정해진 횟수 / '충분' 판단에서 정지

## 행동 궤적까지 반성 (3-LLM)

▶ Self-Refine는 출력 개선, Reflexion은 에이전트 궤적 개선 — 시도를 거듭할수록 똑똑해진다



☞ 반성을 메모리(10차시)에 축적 → 다음 시도에 반영

## 피드백은 자기 자신만이 아니다 (CRITIC)

▶ 9차시(도구)·11차시(RAG)가 반성의 근거로 재등장 — 부품이 연결된다

### 내부 반성

- 자기 출력을 스스로 비평
- 프롬프팅만으로 구현
- Self-Refine · Reflexion

VS

### 외부 반성 (CRITIC)

- 외부 도구(검색 등)로 검증·수정
- 사실 확인의 근거 확보
- 자기비평이 못 잡는 환각을 잡는다

## 무한 반성은 없다 — 멈출 줄도 설계

▶ 반성은 공짜가 아니다

한계

프롬프트 기반 반성은 행동을 유도할 뿐, **파라미터에 각인되진 않는다** (지속 개선=학습/RL)

비용

생성·비평·개선 = 호출 3배 이상 → 지연·요금↑

처방 ①

고위험·고품질이 필요한 곳에 **선별 적용**, 라운드 수 제한

처방 ②

수렴(향상 멈춤)을 관찰하고 **멈추는 것도 설계**

SECTION 03

## Human-Agent 협업 — 사람은 어디서 손을 대나

자율성은 필요한 만큼만. 위험한 행동 앞에는 승인 게이트

## 자율성 ↔ 통제

▶ 윤리·신뢰 원칙: 감독·투명성·설명가능성·공정성·프라이버시

### 완전 자동

- 빠르고 손이 덜 감
- 유연성↑ · 예측가능성↓
- 되돌릴 수 없는 행동엔 위험

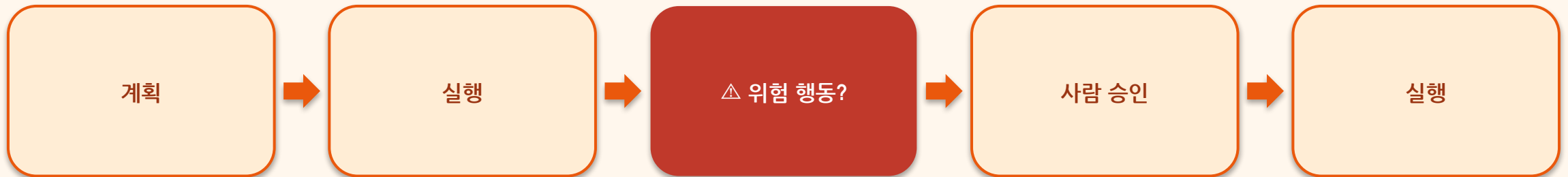
VS

### 사람 승인 게이트

- 고위험·비가역 행동만 게이트
- 감독·투명성·설명가능성
- 신뢰의 기반

## Human-in-the-loop — 위험 행동 전 승인

▶ 비가역·고위험(결제·삭제·외부 발송) 전에 승인 게이트 — 8차시 자율성 슬라이더의 강제 Ask 지점



☞ 거절 시 → 재계획 — 우아한 실패 = 사유 + 대안 + 에스컬레이션

## 신뢰를 주는 에이전트

▶ 능력·한계를 솔직히 + 근거를 투명하게 = 현실적 기대 → 신뢰

### 솔직함

능력·한계·불확실성을  
솔직히 전달

### 투명성

행동 로그·근거(출처) 노출  
— 디버깅·감사도 쉬워진다

### 검증

테스트로 검증  
“테스트 안 된 에이전트는  
신뢰 불가”

### 개입점

언제 끼어들 수 있는지  
UX로 명확히

이 네 기둥 위에 신뢰가 선다 — 신뢰는 우연이 아니라 설계된다 (8차시)



SECTION 04

## 통합 설계 — 내 MVP에 무엇을 붙일까

욕심 금지: 딱 한 기능 + 반성 루프부터

## 좁게 시작하라

▶ ‘고객 만족도 향상’이 아니라 ‘주문 취소 하나를 확실히’

### 모든 걸 자동화 (X)

- 엡지케이스 범람
- 성공 판단 불가(“만족도 향상”)
- 임팩트는 큰데 끝이 안 남

VS

### 한 조각을 확실히 (O)

- 구체적 입력
- 구조화된 출력
- 촘촘한 피드백 루프

## 우리 MVP의 약점 → 붙일 기능 1개

▶ 가장 큰 약점 하나만 고른다 — 기능 1개 + 반성 루프부터

### 도구 / RAG

- 외부 행동·실시간 없음 → 도구(9)
- 헛소리·근거 없음 → RAG(11)
- “말하기”에서 “실제로 바꾸기”로

VS

### 반성 / 계획

- 품질 들쭉날쭉 → 반성
- 다단계를 못 끝냄 → 계획

## 통합 청사진 — Day2 전체를 한 장으로

▶ 계획으로 쪼개고 → 도구/RAG로 근거 있게 → 반성으로 다듬고 → 사람 승인으로 안전하게



🔄 부족하면 루프백 — 충분할 때까지

SECTION 05

# 정리

## 오늘 3줄

- 계획(나눠서) + 반성(고쳐서) + 사람 승인(안전하게) = 끝까지 해내는 에이전트
- Reflection은 품질을 측정 가능하게 올리지만 비용 — 고위험에 선별 적용, 라운드 제한
- Day2의 부품을 내 MVP 한 조각에 끼우면 → 똑똑해진 MVP

**무한 반성은 없다 — 멈출 줄 아는 것도 설계다.**

— 반성은 공짜가 아니다 · 고위험에 선별 적용, 라운드 제한